

Lambda and Recursive Functions

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology, Chennai Campus

premanand.s@vit.ac.in

September 22, 2025

What is a Lambda Function?

- Anonymous function in Python (no name required).
- Created using the `lambda` keyword.
- Syntax:

```
lambda arguments: expression
```

%

%

“begin-frame”[fragile]

“frametitle–Understandings ”

“begin-verbatim”

Understandings

Normal function

```
def square(x):  
    return x * x
```

Lambda equivalent

```
square_lambda = lambda x: x * x
```

```
print(square(5))          # Output: 25
```

```
print(square_lambda(5))   # Output: 25
```

Multiple Arguments

```
add = lambda a, b: a + b  
print(add(3, 7)) # Output: 10
```

Why Use Lambda?

- Small, throwaway functions.
- Useful for map, filter, and sorted.

```
nums = [1, 2, 3, 4, 5]
```

```
# Using map
```

```
squares = list(map(lambda x: x*x, nums))  
print(squares)  # [1, 4, 9, 16, 25]
```

```
# Using filter
```

```
evens = list(filter(lambda x: x % 2 == 0, nums))  
print(evens)  # [2, 4]
```

Sorting with Lambda

```
students = [("Alice", 25), ("Bob", 20), ("Charlie", 23)]

# Sort by age
students_sorted = sorted(students, key=lambda x: x[1])

print(students_sorted)
# [('Bob', 20), ('Charlie', 23), ('Alice', 25)]
```

Limitations of Lambda

- Single expression only.
- Cannot contain statements (e.g., loops, print).
- Readability suffers if overused.

When to Use

- Small, temporary functions.
- One-liners inside `map`, `filter`, or `sorted`.
- Not a replacement for full `def` functions.

Basic Lambda Questions

- 1 Write a lambda function to calculate the square of a number.
- 2 Write a lambda function to check if a number is even.
- 3 Write a lambda function that returns the maximum of two numbers.
- 4 Convert the normal function `def add(a,b): return a+b` into a lambda.

Lambda with Built-in Functions

- 1 Use `map()` with a lambda to cube all numbers in a list.
- 2 Use `filter()` with a lambda to get only numbers greater than 10.
- 3 Use `sorted()` with a lambda to sort words by their last letter.
- 4 Use `reduce()` (from `functools`) with a lambda to find the product of all numbers in a list.

Applied Lambda Challenges

- ① You have a list of tuples [(‘‘Alice’’, 25), (‘‘Bob’’, 19), (‘‘Charlie’’, 30)].
 - Sort by name.
 - Sort by age.
- ② From a list of names, keep only those that start with the letter “A”.
- ③ Write a lambda to reverse a string.
- ④ Use `map()` with lambda to convert a list of temperatures in Celsius to Fahrenheit.

Thought-Provoking Lambda Problems

- 1 Create a lambda function that returns `True` if a number is both even and divisible by 5.
- 2 Given a list of numbers, use `map()` and a lambda to label them as “Even” or “Odd”.
- 3 Write a lambda function that calculates the factorial of a number using `reduce()`.
- 4 Discussion: When should you avoid using lambda and prefer normal `def` functions?

What is Recursion?

- A **recursive function** is a function that calls itself.
- Useful for solving problems that can be broken into smaller, similar sub-problems.
- Every recursive function must have:
 - 1 **Base Case**: stopping condition.
 - 2 **Recursive Case**: function calling itself on smaller input.

Structure of a Recursive Function

```
def function(parameters):  
    if base_case_condition:  
        return result  
    else:  
        return function(smaller_input)
```

Example 1: Factorial

```
def factorial(n):  
    if n == 0 or n == 1:    # Base case  
        return 1  
    return n * factorial(n-1) # Recursive case  
  
print(factorial(5)) # 120
```


Example 2: Fibonacci

```
def fibonacci(n):  
    if n <= 1:    # Base case  
        return n  
    return fibonacci(n-1) + fibonacci(n-2)  
  
print(fibonacci(6))    # 8
```

How Recursion Works (Call Stack)

- Each recursive call is stored in the **call stack**.
- The function is paused until the smaller call returns.
- Example: `factorial(3)`
 - 1 `factorial(3) → 3 * factorial(2)`
 - 2 `factorial(2) → 2 * factorial(1)`
 - 3 `factorial(1) = 1` (base case)
 - 4 Stack unwinds: $3 * 2 * 1 = 6$

Example 3: Sum of List

```
def list_sum(nums):  
    if len(nums) == 0:    # Base case  
        return 0  
    return nums[0] + list_sum(nums[1:])  
  
print(list_sum([1, 2, 3, 4]))    # 10
```

Example 4: Reverse String

```
def reverse_string(s):  
    if s == "":  
        return ""  
    return reverse_string(s[1:]) + s[0]  
  
print(reverse_string("hello"))  # "olleh"
```

Example 5: Palindrome Check

```
def is_palindrome(s):  
    if len(s) <= 1:  
        return True  
    if s[0] != s[-1]:  
        return False  
    return is_palindrome(s[1:-1])  
  
print(is_palindrome("madam"))  # True
```

Example 6: Tower of Hanoi

```
def hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    hanoi(n-1, source, auxiliary, target)
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n-1, auxiliary, target, source)

hanoi(3, "A", "C", "B")
```

Key Points to Remember

- Always define a **base case**.
- Recursive calls must reduce the problem size.
- Python has a recursion depth limit (default: 1000).
- Iteration is usually more efficient, but recursion is elegant for:
 - Divide-and-conquer algorithms (Merge Sort, Quick Sort)
 - Tree and graph traversal (DFS)
 - Backtracking problems (N-Queens, Sudoku)

Practice Problems on Recursion

Solve the following problems using recursion

Problem 1: Factorial

Task

Write a recursive function to calculate the factorial of a number n .

Example: $\text{factorial}(5) = 120$

Problem 2: Sum of Digits

Task

Write a recursive function to calculate the sum of digits of a number.

Example: `sum_digits(1234)` = 10

Problem 3: Power Function

Task

Write a recursive function to calculate a^b (a raised to the power b).

Example: `power(2, 5) = 32`

Problem 4: Greatest Common Divisor (GCD)

Task

Write a recursive function to compute the GCD of two numbers using the Euclidean algorithm. **Example:** $\text{gcd}(48, 18) = 6$

Problem 5: Reverse String

Task

Write a recursive function to reverse a given string. **Example:**
`reverse("hello") = "olleh"`

Problem 6: Palindrome Check

Task

Write a recursive function to check whether a string is a palindrome.

Example: `is_palindrome("madam") = True`

Problem 7: Fibonacci Sequence

Task

Write a recursive function to generate the n^{th} Fibonacci number.

Example: `fibonacci(6) = 8`

Problem 8: Count Digits

Task

Write a recursive function to count the number of digits in an integer.

Example: `count_digits(12345)` = 5

Problem 9: Sum of List Elements

Task

Write a recursive function to find the sum of all elements in a list.

Example: `list_sum([1,2,3,4]) = 10`

Problem 10: Tower of Hanoi

Task

Solve the Tower of Hanoi problem for n disks using recursion. **Example:** Show the steps to move 3 disks from Source to Target.